CSA02 – ASSIGNMENT 1

Grama Jyothi Rural Electricity Co-operative

Tasks 1–4: Headers, Input/Output, Type Conversion and Compilation

Name: RAKSHITHA R

Register Number: 192525454

Course: CSA0209

GitHub Repository Link: https://github.com/RAKSHITHA-5656/CP_CSA0209

Scenario Overview

Grama Jyothi calculates electricity bills using previous and current meter readings. The program must calculate consumption, apply slab rates, add the fixed charge, print accurate amounts to the paisa, validate input, handle meter rollover, and support growth from 4,000 to 800,000 households.

Example tariff: first 100 units free; units 101–200 at Rs 2.25 per unit; units above 200 at Rs 4.50 per unit; fixed charge Rs 50.00. Money is stored as integer paise.

TASK 1 – Organise the Program with Header Files and Conditional Compilation (3 Marks)

1.1 File Structure

```
Task01/\n  billing.h\n  billing.c\n  main.c\n  output_normal.txt\n  output_debug.txt
```

1.2 Header File: billing.h

```
#ifndef BILLING_H
#define BILLING_H

#define SLAB1_UNITS 100
#define SLAB2_UNITS 200
#define RATE1_PAISE 0L
#define RATE2_PAISE 225L
#define RATE3_PAISE 450L
#define FIXED_CHARGE_PAISE 5000L

typedef unsigned short meter_t;
unsigned int consumption(meter_t previous, meter_t current);
long compute_bill(unsigned int units);

#endif
```

1.3 Implementation File: billing.c

```
#include <stdio.h>
#include "billing.h"

unsigned int consumption(meter_t previous, meter_t current)
{
    unsigned int units = (unsigned short)(current - previous);
#ifdef DEBUG
    printf("[DEBUG] previous=%u current=%u units=%u\\n",
           (unsigned)previous, (unsigned)current, units);
#endif
    return units;
}

long compute_bill(unsigned int units)
{
    long paise = FIXED_CHARGE_PAISE;
    unsigned int remaining = units;
    if (remaining > SLAB2_UNITS) {
        paise += (long)(remaining - SLAB2_UNITS) * RATE3_PAISE;
        remaining = SLAB2_UNITS;
    }
    if (remaining > SLAB1_UNITS)
        paise += (long)(remaining - SLAB1_UNITS) * RATE2_PAISE;
#ifdef DEBUG
    printf("[DEBUG] units=%u bill_paise=%ld\\n", units, paise);
#endif
```

```
    return paise;
}
```

```
#ifndef BILLING_H
#define BILLING_H

typedef unsigned short meter_t;

#define SLAB1_UNITS 100
#define SLAB2_UNITS 200

#define RATE2_PAISE 225
#define RATE3_PAISE 450

#define FIXED_CHARGE_PAISE 5000

unsigned int consumption(meter_t previous, meter_t current);
long compute_bill(unsigned int units);

#endif
#include <stdio.h>

typedef unsigned short meter_t;

#define SLAB1_UNITS 100
#define SLAB2_UNITS 200

#define RATE2_PAISE 225
#define RATE3_PAISE 450

#define FIXED_CHARGE_PAISE 5000

unsigned int consumption(meter_t previous, meter_t current)
```

```
input
Units consumed: 330
Bill: Rs 860.00


...Program finished with exit code 0
Press ENTER to exit console.
```

1.4 Main Program: main.c

```
#include <stdio.h>
#include "billing.h"

int main(void)
{
    meter_t previous = 1200, current = 1530;
    unsigned int units = consumption(previous, current);
    long bill = compute_bill(units);
    printf("Units consumed: %u\\n", units);
    printf("Bill: Rs %ld.%02ld\\n", bill / 100, bill % 100);
    return 0;
}
```

Expected Output

```
Units consumed: 330\nBill: Rs 635.00
```

Compile normally: gcc -std=c11 -Wall -Wextra main.c billing.c -o billing. Debug build: gcc -std=c11 -Wall -Wextra -DDEBUG main.c billing.c -o billing_debug. Run ./billing or ./billing_debug (on Windows, use the corresponding .exe name).

1.5 Explanation

- Header files contain shared constants, type definitions and function prototypes. Source files contain function definitions; main() controls program execution.
- Include guards (#ifndef, #define, #endif) prevent repeated inclusion of the same header in one translation unit.
- #include <file> searches standard/system include paths; #include "file" searches the source/current directory first, then configured include paths.
- #ifdef DEBUG includes diagnostic statements only when compiled with -DDEBUG.

Alternative: Run-Time Debug Variable

```
int debug = 1;
if (debug) {
    printf("Intermediate value: %u\\n", units);
}
```

A run-time debug variable adds an O(1) test each time the debug point is reached. Debug code remains in the executable unless optimized away. Without -DDEBUG, preprocessor-controlled statements are removed before compilation, so they add no run-time test and normally no executable code. With a fixed number of debug points, the extra space is O(1); compile-time exclusion can reduce executable size.

TASK 2 – Read and Print Data Correctly (4 Marks)

2.1 Format Specifiers

Data type	printf	scanf
int	%d	%d
unsigned int	%u	%u
long	%ld	%ld
long long	%lld	%lld
float	%f	%f
double	%f	%lf
char	%c	%c
string (char array)	%s	%s

For printf, double uses %f; for scanf, double uses %lf and float uses %f. A mismatched format specifier can produce incorrect output or undefined behaviour.

2.2 Input Validation Example

```
#include <stdio.h>
#include "billing.h"

int main(void)
{
    unsigned int household, category, previous, current;
    unsigned int units;
    long bill;

    printf("Household number: ");
    if (scanf("%u", &household) != 1) {
        fprintf(stderr, "Invalid household number.\\n"); return 1;
    }
    printf("Category number: ");
    if (scanf("%u", &category) != 1) {
        fprintf(stderr, "Invalid category.\\n"); return 1;
    }
    printf("Previous and current readings: ");
    if (scanf("%u %u", &previous, &current) != 2 ||
        previous > 65535U || current > 65535U) {
        fprintf(stderr, "Invalid meter readings.\\n"); return 1;
    }

    units = (unsigned short)(current - previous);
    bill = compute_bill(units);
    printf("Household: %u, category: %u\\n", household, category);
    printf("Units consumed: %u\\n", units);
    printf("Bill: Rs %ld.%02ld\\n", bill / 100, bill % 100);
    return 0;
}
```

```
Category number: 1
Previous and current readings: 1200 1530
Household: 101, category: 1
Units consumed: 330
Bill: Rs 860.00
```

Example: household 101, category 1, previous 1200, current 1530 gives 330 units and Rs 860.00. Category is read for demonstration; this example uses one tariff for every category.

2.3 Rupees and Paise

If a bill is 16250 paise, bill / 100 is 162 rupees and bill % 100 is 50 paise. Print with printf("Rs %ld.%02ld", bill / 100, bill % 100) to show two digits after the decimal point.

2.4 Wrong Format Specifier Test

```
#include <stdio.h>
int main(void)
{
    long value = 3000000000L;
    printf("Correct: %ld\\n", value);
    /* Incorrect format example: printf("%d\\n", value); */
    return 0;
}
```

Use %ld for long. Using %d for a long is a format mismatch and causes undefined behaviour; output can be wrong or appear correct by accident. Keep the incorrect line commented out in the submitted working program. Compile with -Wall -Wextra to see diagnostics where detectable.

2.5 Alternative: fgets and strtol

```
#include <stdio.h>
#include <stdlib.h>
#include <errno.h>
#include <limits.h>

int main(void)
{
    char line[128], *end;
    long value;
    printf("Enter household number: ");
    if (fgets(line, sizeof line, stdin) == NULL) {
        fprintf(stderr, "No input received.\\n"); return 1;
    }
    errno = 0;
    value = strtol(line, &end, 10);
    if (end == line || errno == ERANGE) {
        fprintf(stderr, "Invalid or out-of-range number.\\n"); return 1;
    }
    while (*end == ' ' || *end == '\\t' || *end == '\\n' || *end == '\\r')
        end++;
    if (*end != '\\0' || value < 0 || value > UINT_MAX) {
        fprintf(stderr, "Invalid household number.\\n"); return 1;
    }
    printf("Household number: %ld\\n", value);
    return 0;
}
```

```
Enter household number: 101
Household number: 101


...Program finished with exit code 0
Press ENTER to exit console.
```

For a line of length L, reading and parsing take O(L) time. A fixed buffer of B bytes uses O(B) space; here B = 128. fgets allows the program to inspect a complete line and recover by reporting or discarding invalid input. scanf is shorter but can leave invalid characters in the input stream.

TASK 3 – Control Type Conversion and Precision (4 Marks)

3.1 Conversion Points

- unsigned short meter readings may be promoted to int before subtraction; cast back to the 16-bit unsigned type to obtain modulo-65536 rollover behaviour.
- Multiplying units by a tariff rate can overflow a narrow type; use a sufficiently wide type before multiplication.
- Integer division by 100 removes the fractional part; use quotient and remainder only when formatting paise as rupees and paise.
- Casting a long value outside the range of int can lose information; do not cast unless the value is known to fit.
- If fractional paisa occurs, document one rounding rule, such as round to nearest paisa (half-up) once per bill.

3.2 Unexpected Expressions

```
#include <stdio.h>
int main(void)
{
    printf("1) 5 / 2 = %d\\n", 5 / 2);
    printf("2) (double)(5 / 2) = %.1f\\n", (double)(5 / 2));
    printf("3) (double)5 / 2 = %.1f\\n", (double)5 / 2);
    float f = 0.1f;
    printf("4) 0.1f * 10 = %.9f\\n", f * 10);
    long big = 3000000000L;
    printf("5) long value = %ld\\n", big);
    /* Casting big to int may be implementation-defined if it cannot fit. */
    return 0;
}
```

```
1) 5 / 2 = 2\n2) (double)(5 / 2) = 2.0\n3) (double)5 / 2 = 2.5\n4) 0.1f * 10 = 1.000000000\n5) long value = 3000000000\n

...Program finished with exit code 0
Press ENTER to exit console.
```

Explanation

- 5 / 2 prints 2 because integer division discards the fractional part.
- (double)(5 / 2) prints 2.0 because division occurs before the cast.
- (double)5 / 2 prints 2.5 because conversion occurs before division.
- 0.1f is not exactly representable in binary floating-point, so small precision differences may appear.
- A long-to-int conversion can lose information if the value is outside the int range.

3.3 Float Versus Integer Paise

```
#include <stdio.h>
int main(void)
{
    float total_float = 0.0f;
    long total_paise = 0;
    for (int i = 0; i < 4000; i++) {
```

```
        total_float += 0.1f;
        total_paise += 10;
    }
    printf("float total = %.7f rupees\\n", total_float);
    printf("integer total = Rs %ld.%02ld\\n",
           total_paise / 100, total_paise % 100);
    return 0;
}
```

```
float total = 400.0151978 rupees\ninteger total = Rs 400.00\n

...Program finished with exit code 0
Press ENTER to exit console.
```

The integer total is exactly Rs 400.00. The float result may differ slightly due to binary representation and repeated additions. Run for n = 100, 1,000, 4,000 and 100,000 and record actual output; error depends on the values and operations and does not necessarily grow at a perfectly constant rate.

Feature	Integer paise	float rupees
Addition time	O(1)	O(1)
Typical size	long: 4 or 8 bytes; long long usually 8	Usually 4 bytes
Exactness	Exact integer paise within range	Approximate decimal representation
Risk	Integer overflow if range exceeded	Rounding error and limited significant digits

Adding n bills to a running total takes O(n) time and O(1) extra space. Storing all n bills requires O(n) space.

TASK 4 – Trace the Compilation Process (4 Marks)

4.1 Compilation Commands

Assume the program uses main.c and billing.c with billing.h shared between them.

```
# 1. Preprocess
gcc -E main.c -o main.i
gcc -E billing.c -o billing.i

# 2. Generate assembly
gcc -S main.c -o main.s
gcc -S billing.c -o billing.s

# 3. Create object files
gcc -c main.c -o main.o
gcc -c billing.c -o billing.o

# 4. Link
gcc main.o billing.o -o billing

# 5. Build with warnings
gcc -std=c11 -Wall -Wextra main.c billing.c -o billing
```

Stage	Output	Purpose
Preprocessing (-E)	.i / expanded text	Expands macros, includes headers, processes conditional compilation.
Compilation (-S)	.s assembly	Translates C source into assembly.
Assembly (-c)	.o object file	Creates machine code; external references may remain unresolved.
Linking	Executable	Combines object files/libraries and resolves external symbols.

4.2 Preprocessor Changes

- The contents of billing.h are included after include guards are processed.
- #define constants are substituted into expressions.
- With -DDEBUG, #ifdef DEBUG blocks are included; without it, those statements are removed before compilation.
- Comments are removed during preprocessing.

4.3 Error Categories

Type	Example	Explanation
Compile-time	Missing semicolon or misspelled keyword	Compiler cannot translate the source successfully.
Link-time	Call compute_bill() but omit billing.o at link time	Linker cannot find the function definition.
Run-time	Enter invalid input and the program reports an error and exits	The program is executing and handles a run-time input problem.

Use a safe invalid-input test as run-time evidence; do not deliberately submit unsafe operations.

4.4 Warnings

gcc -Wall -Wextra can report format mismatches, unused variables or parameters, and suspicious code. Resolve warnings by using the correct format specifier, removing or using unused variables, and matching declarations with definitions. Save a screenshot of a warning-free build if possible.

4.5 Alternative: Multiple Source Files

```
gcc -std=c11 -Wall -Wextra -c main.c -o main.o
gcc -std=c11 -Wall -Wextra -c input.c -o input.o
gcc -std=c11 -Wall -Wextra -c tariff.c -o tariff.o
gcc -std=c11 -Wall -Wextra -c report.c -o report.o
gcc main.o input.o tariff.o report.o -o billing
```

Feature	One source file	Multiple source files
Change to one function	Usually recompile the entire source file	Usually recompile only changed .c file, then relink
Build time	Can be longer for large source	Often shorter for large projects
Object-file space	One object file	Several object files; temporary total may be larger
Organization	Simple for small programs	Better separation and maintenance

If a shared header changes, every source file that includes it may need recompilation. Exact build time depends on source size, dependencies, compiler options and hardware.

Design Challenge: Smart Meters and Time-of-Day Tariffs

The design should be modified rather than retained unchanged. Integer paise, input validation, header files and rollover handling remain useful, but 15-minute readings and time-of-day tariffs require a new interval-based data model and tariff engine.

- Store timestamped interval readings, household ID, meter value and validation status.
- Calculate interval consumption with rollover checks and detect missing or duplicate readings.
- Store tariff periods and rates as configuration data instead of hard-coding one rate.
- Use wide integer types for accumulated units and money; check overflow and document rounding policy.
- Separate input, validation, tariff selection, billing and reports into modules.
- For district scale, process records in batches and maintain audit logs so bills can be reproduced.

Conclusion: retain integer money and safe rollover logic, but redesign the data model and tariff engine for timestamped intervals, configurable rates and scalable processing.

GitHub Submission Checklist

- Create repository: CSA02-<RegisterNumber>-Assignment1.
- Create folders Task01, Task02, Task03 and Task04.
- Include source code, input files where relevant, output screenshots and compilation evidence.
- README.md must include name, register number, compiler/version, exact GCC commands, run steps and results.
- Make at least one commit for each task and push progressively.
- Share the repository publicly or add the faculty as a collaborator.
- Put the working GitHub link on the cover page and in README.md.

Suggested README.md Template

```
# CSA02 – Assignment 1
- Name: [Your name]
- Register number: [Your register number]
- Compiler: GCC [version from gcc --version]
- Repository: [GitHub link]

## Task 1
Files: billing.h, billing.c, main.c
Compile: gcc -std=c11 -Wall -Wextra main.c billing.c -o billing
Debug: gcc -std=c11 -Wall -Wextra -DDEBUG main.c billing.c -o billing_debug
Run: ./billing (Windows: billing.exe)
Result: [Brief result and screenshot filename]

## Task 2
Compile: gcc -std=c11 -Wall -Wextra input_bill.c billing.c -o input_bill
Run: ./input_bill
Result: [Input validation and formatted bill]

## Task 3
Compile: gcc -std=c11 -Wall -Wextra precision.c -o precision
Run: ./precision
Result: [Record conversion and precision observations]

## Task 4
Preprocess: gcc -E main.c -o main.i
Assembly: gcc -S main.c -o main.s
Object: gcc -c main.c -o main.o
Link: gcc main.o billing.o -o billing
Result: [Compilation evidence and error/warning classification]
```

Before submission, replace placeholders, make filenames and commands match your actual files, execute every program yourself, record your compiler version and outputs, and add your own screenshots.